

# OOP Practical Sessions

*BSc IT - Object-Oriented Programming with Java*

# DAY 1 & 2 - Practical Session

## E-Commerce Product Management System

### Q1 Designing the Product Class

#### Background / Scenario

You have just joined a startup called **ShopEasy**, an online retail platform. The development team needs you to model a **Product** entity that represents any item sold on the platform.

Every product on ShopEasy has a **product ID**, a **name**, a **price**, a **stock quantity**, and a **category** (e.g., Electronics, Clothing, Books). The system must be able to create products both **with and without initial data** — for example, a **placeholder product** might be created first and filled in later.

#### What You Must Do

#### Tasks

- ▶ Declare a class called **Product** with five instance variables: productId (int), name (String), price (double), stockQuantity (int), category (String).
- ▶ Write a **default (no-argument) constructor** that sets productId to 0, name to "Unknown", price to 0.0, stockQuantity to 0, and category to "Uncategorized".
- ▶ Write a parameterized constructor that accepts all five values and assigns them using the this keyword to distinguish fields from parameters.
- ▶ Write a method called displayProduct() that prints all five fields in a neat, labelled format.
- ▶ In a Main class, create one object using the default constructor, one using the parameterized constructor, call displayProduct() on both, and observe the difference.

#### Expected Output

```
===== Default Product =====
Product ID   : 0
Name         : Unknown
Price        : $0.0
Stock        : 0 units
Category     : Uncategorized

===== Parameterized Product =====
Product ID   : 101
Name         : Wireless Headphones
Price        : $49.99
Stock        : 150 units
Category     : Electronics
```

## ✓ Answer — Product.java

```
public class Product {                                // Class declaration –
                                                        // defines the blueprint

    public int productId;                             // private: only accessible
                                                        // inside this class

    public String name;                               //String holds text-product
                                                        // name

    public double price;                              // double for decimal prices

    public int stockQuantity;                         // number of units available

    public String category;                           // product category label

    // — Default Constructor —————
    public Product() {                                // Called when no arguments
                                                        // are given
        this.productId    = 0;                       // 'this.' makes clear we
                                                        // mean the field

        this.name         = "Unknown";

        this.price        = 0.0;

        this.stockQuantity = 0;

        this.category     = "Uncategorized";
    }

    // — Parameterized Constructor —————
    public Product(int productId, String name,
                    double price, int stockQuantity,
                    String category) {
        this.productId    = productId;                // Parameters share names
                                                        // with fields

        this.name         = name;

        this.price        = price;

        this.stockQuantity = stockQuantity;

        this.category     = category;
    }

    // — Display Method —————
    public void displayProduct() {                    // void: returns nothing –
                                                        // just prints

        //System.out.println("==== Product Details =====");
        System.out.println("Product ID    : " + productId); // Java auto-converts int to
                                                        // String with +

        System.out.println("Name          : " + name);

        System.out.println("Price         : $" + price);

        System.out.println("Stock         : " + stockQuantity + "
units");
        System.out.println("Category      : " + category);

    }
}
```

## ✓ Answer — Main.java

```
public class Main {  
    public static void main(String[] args) {  
        // Entry point of every Java  
        // program  
  
        Product p1 = new Product();  
        // Calls the default  
        // constructor – p1 on stack,  
        // object on heap  
  
        Product p2 = new Product(  
            // Calls the parameterized  
            // constructor  
            101, "Wireless Headphones",  
            49.99, 150, "Electronics");  
  
        System.out.println("==== Default Product ====");  
        p1.displayProduct();  
        // Calls method on p1 –  
        // 'this' inside method = p1  
  
        System.out.println("");  
        System.out.println("==== Parameterized Product  
====");  
        p2.displayProduct();  
        // Calls method on p2 –  
        // 'this' inside method = p2  
        System.out.println();  
    }  
}
```

## Q2

## Discount & Pricing Logic — Method Overloading

### Background / Scenario

ShopEasy's marketing team wants to introduce flexible discount campaigns. Discounts can be applied in three different ways depending on the campaign type:

**Campaign A — Seasonal Sale:** A flat 10% discount is applied to everything, no input needed.

**Campaign B — Custom Discount:** The marketing team specifies exactly what percentage to cut.

**Campaign C — Bulk Discount:** A discount is applied only if the stock level is above a given threshold — the store only discounts items they have plenty of.

All three campaigns use the same method name `applyDiscount()` because conceptually they all 'apply a discount'. Java's method overloading allows this: same name, different parameters.

### What You Must Do

#### Tasks

- ▶ Add three overloaded versions of `applyDiscount()` to your `Product` class from Question 1.
- ▶ Version 1 — `applyDiscount()`: no parameters. Applies exactly 10% discount. Prints the new price.
- ▶ Version 2 — `applyDiscount(double discountPercent)`: one parameter. Applies the specified percentage. Validates that the percentage is between 1 and 99 — print an error message if invalid.
- ▶ Version 3 — `applyDiscount(double discountPercent, int minStock)`: two parameters. Applies the discount only if current `stockQuantity` is greater than or equal to `minStock`. Otherwise prints 'Stock too low for this discount.'
- ▶ In Main, create a `Product` with price 100.00 and stock 80. Call all three versions and print the price after each call to observe how it changes.

 **NOTE:** After three discounts on the same object the price keeps falling from the previous value. Make sure you notice this — state is persistent inside an object.

#### Expected Output

```
Default 10% applied.  New price: $90.0
Custom 15% applied.   New price: $76.5
Stock OK. 20% applied. New price: $61.2

Stock too low for this discount.
```

## ✓ Answer — add to Product.java

```
// — Overload 1: no parameter —————
public void applyDiscount() {
    price = price - (price * 0.10);
    System.out.println("Default 10% applied. New price: $" +
price);
}

// — Overload 2: custom percentage —————
public void applyDiscount(double discountPercent) {
    if (discountPercent < 1 || discountPercent > 99) {
        System.out.println("Invalid discount: " +
discountPercent + "%");
        return;
    }
    price = price - (price * discountPercent / 100);
    System.out.println("Custom " + discountPercent + "%
applied. New price: $" + price);
}

// — Overload 3: bulk discount with stock check —
public void applyDiscount(double discountPercent, int
minStock) {
    if (stockQuantity >= minStock) {

        price = price - (price * discountPercent / 100);
        System.out.println("Stock OK. " + discountPercent +
"% applied. New price: $" + price);
    } else {
        System.out.println("Stock too low for this
discount.");
    }
}
```

```
// Called with zero arguments
// Reduce price by 10%
```

```
// One argument – the % to cut
// Validate before applying
```

```
// Exit method early – do not apply
The return; here is called an early
exit or guard clause. It stops the
method immediately when the input
is invalid, so the lines below it
never execute.
```

```
// Two arguments
```

```
// Business rule: only discount if
enough stock
```

## ✓ Answer — Main.java calls

```
Product tv = new Product(202, "Smart TV", 100.00, 80,
"Electronics");
```

```
tv.applyDiscount();
```

```
tv.applyDiscount(15);
```

```
// Compiler picks Overload 1
– no args
```

```
// Compiler picks Overload 2
– one double
```

```

tv.applyDiscount(20, 50);                                     // Compiler picks Overload 3
                                                             - double + int. Stock 80 >=
                                                             50 → applies

tv.applyDiscount(20, 90);                                     // Stock 80 < 90 → 'Stock too
System.out.println();                                         low' message

```

 **TIP:** Method overloading is resolved at COMPILE TIME. The compiler looks at the arguments you pass and decides which overload to call. This is why it is also called static polymorphism or early binding.

After three discounts on the same object the price keeps falling from the previous value. How to avoid price falling in question 02.

The issue is that `applyDiscount()` permanently modifies the price field each time it's called. There are two clean ways to fix this:

Store the `originalPrice` separately when the object is created, and always calculate discount *from the original*, not the current price. Best when you want to allow re-pricing but always relative to the starting price.

**This is the more realistic and teachable approach** for an e-commerce system — a product has a listed price, and any discount is always a reduction from that listed price, not from a previously discounted price.

```

public class Product {
    private int    productId;
    private String name;
    private double price;           // current/displayed price
    private double originalPrice;   // locked in at creation — never changed
    private int    stockQuantity;
    private String category;

    public Product(int productId, String name, double price,
                   int stockQuantity, String category) {
        this.productId    = productId;
        this.name          = name;
        this.price         = price;
        this.originalPrice = price; // snapshot of the starting price
        this.stockQuantity = stockQuantity;
        this.category      = category;
    }

    // Overload 1 — flat 10%, always from originalPrice
    public void applyDiscount() {
        this.price = originalPrice - (originalPrice * 0.10);
        System.out.println("10% discount applied. Sale price: $" + this.price);
    }

    // Overload 2 — custom %, always from originalPrice
    public void applyDiscount(double discountPercent) {
        if (discountPercent < 1 || discountPercent > 99) {
            System.out.println("Invalid discount: " + discountPercent + "%");
            return;
        }
        this.price = originalPrice - (originalPrice * discountPercent / 100);
        System.out.println(discountPercent + "% discount applied. Sale price: $" + this.price);
    }
}

```

```
// Overload 3 — bulk discount, always from originalPrice
public void applyDiscount(double discountPercent, int minStock) {
    if (stockQuantity >= minStock) {
        this.price = originalPrice - (originalPrice * discountPercent / 100);
        System.out.println("Bulk " + discountPercent + "% applied. Sale price: $" + this.price);
    } else {
        System.out.println("Stock too low for this discount.");
    }
}

// Reset back to the original listed price
public void resetPrice() {
    this.price = originalPrice;
    System.out.println("Price reset to original: $" + originalPrice);
}
}
```



## Q3

## Specialised Products — Inheritance, Overriding & Access Modifiers

### Background / Scenario

ShopEasy sells not only physical goods but also digital downloads — eBooks, software licences, and online courses. A digital product IS-A product (it has all the same basic attributes) but it also has unique properties: a download URL and a file size in MB.

The architecture decision is to use inheritance: DigitalProduct will extend Product. This avoids duplicating the common fields (productId, name, price, etc.) while letting DigitalProduct add its own specialised data.

### What You Must Do

#### Tasks

- ▶ Create a new class DigitalProduct that extends Product.
- ▶ Add two new private fields to DigitalProduct: downloadUrl (String) and fileSizeMB (double).
- ▶ Write a constructor in DigitalProduct that accepts all 7 values (5 from Product + 2 new). Use super() to pass the first 5 to the Product constructor. super() must be the FIRST line.
- ▶ Override the displayProduct() method in DigitalProduct. Use super.displayProduct() to print the inherited fields first, then print the two new fields.
- ▶ Add the @Override annotation above the overriding method.
- ▶ In Main, create a DigitalProduct object and call displayProduct(). Confirm that both the parent and child fields are printed.

#### Expected Output

```
Product ID   : 301
Name         : Java Programming eBook
Price        : $24.99
Stock        : 9999 units
Category     : Books
--- Digital Details ---
Download URL : https://shopEasy.com/downloads/java-ebook
File Size    : 8.5 MB
```

## ✓ Answer — DigitalProduct.java

```
public class DigitalProduct extends Product {  
    // IS-A relationship:  
    // DigitalProduct IS-A Product  
  
    private String downloadUrl;  
    // private: new fields belong  
    // only to DigitalProduct  
  
    private double fileSizeMB;  
  
    public DigitalProduct(int productId, String name,  
                           double price, int stock,  
                           String category,  
                           String downloadUrl, double  
fileSizeMB) {  
        super(productId, name, price, stock, category);  
        // MUST be first line – calls  
        // Product constructor  
        this.downloadUrl = downloadUrl;  
        // Then initialise the new  
        // fields  
        this.fileSizeMB = fileSizeMB;  
    }  
  
    @Override  
    // Tells compiler:  
    // intentionally replacing  
    // parent version  
    public void displayProduct() {  
        // Same signature as in  
        // Product – this IS an override  
        // Calls Product's version  
        // first – reuses parent logic  
        super.displayProduct();  
  
        System.out.println("--- Digital Details ---");  
        System.out.println("Download URL : " + downloadUrl);  
        System.out.println("File Size      : " + fileSizeMB + "  
MB");  
    }  
  
    public String getDownloadUrl() { return downloadUrl; }  
    // public getter for private  
    // field  
  
    public double getFileSizeMB() { return fileSizeMB; }  
}
```

## ✓ Answer — Main.java

```
DigitalProduct dp = new DigitalProduct(  
    // Reference type =  
    // DigitalProduct  
    301, "Java Programming eBook", 24.99, 9999, "Books",  
    "https://shopEasy.com/downloads/java-ebook", 8.5);  
dp.displayProduct();  
// Calls overriding version  
// in DigitalProduct  
  
Product p = dp;  
// Parent reference pointing  
// to child object
```

```
p.displayProduct();
```

```
// Still calls  
DigitalProduct's version –  
runtime polymorphism
```

 **TIP:** Key question: 'What is the difference between overloading and overriding?' Overloading = same class, same name, different parameter list, resolved at compile time. Overriding = subclass replaces parent method, same signature, resolved at runtime.

# DAY 2 & 3 - Practical Session

## Hospital Staff & Patient Management System

### Q4

### Patient Registration - this Keyword & Constructor Chaining



#### Background / Scenario

City General Hospital's IT team is building a patient registration module. A Patient record holds: `patientId`, `name`, `age`, `ward`, and a `boolean isAdmitted` flag.

The registration desk encounters two scenarios:

**Full registration:** All details are available immediately — `patientId`, `name`, `age`, and `ward` are all provided.

**Quick registration:** Only `patientId` and `name` are known at the door (e.g., emergency intake). `Age` defaults to 0 and `ward` defaults to 'General'.

After registration, `ward` and `age` can be updated one at a time via setters. The team wants to support fluent-style updates, e.g.: `patient.setAge(45).setWard("ICU").printInfo()`.



#### What You Must Do

##### Tasks

- ▶ Create a `Patient` class with five fields: `patientId` (int), `name` (String), `age` (int), `ward` (String), `isAdmitted` (boolean).
- ▶ Write a full 4-argument constructor (`patientId`, `name`, `age`, `ward`). Inside it, use this keyword to resolve the name-clash between fields and parameters. Set `isAdmitted` to true by default.
- ▶ Write a 2-argument constructor (`patientId`, `name`) that uses `this(patientId, name, 0, "General")` to chain to the full constructor. Do NOT duplicate the assignment logic.
- ▶ Write `setAge(int age)` and `setWard(String ward)` that both return `Patient` (return this) to enable method chaining.
- ▶ Write a `printInfo()` method that prints all five fields.
- ▶ In Main: create one patient using the full constructor, and one using the 2-arg constructor. On the second patient, chain `setAge` and `setWard` before printing.

##### Expected Output

```
Patient ID : 1001 | Name: Alice Brown
Age: 35 | Ward: ICU | Admitted: true
```

```
Patient ID : 1002 | Name: Bob Perera
Age: 47 | Ward: Cardiology | Admitted: true
```

## ✓ Answer — Patient.java

```
public class Patient {
    private int    patientId;
    private String name;
    private int    age;
    private String ward;
    private boolean isAdmitted;

    // — Full Constructor —————
    public Patient(int patientId, String name,
                   int age, String ward) {
        this.patientId = patientId;

        this.name      = name;
        this.age       = age;
        this.ward      = ward;
        this.isAdmitted = true;
    }

    // — Short Constructor: chains to full —————
    public Patient(int patientId, String name) {
        this(patientId, name, 0, "General");
    }

    // — Method-chaining setters —————
    public Patient setAge(int age) {
        this.age = age;
        return this;
    }

    public Patient setWard(String ward) {
        this.ward = ward;
        return this;
    }

    public void printInfo() {
        System.out.println("Patient ID : " + this.patientId + "
| Name: " + this.name);
        System.out.println("Age: "      + this.age + " |
Ward: " + this.ward
                        + " | Admitted: " +
this.isAdmitted);
    }
}
```

// 4 params – all details known

// 'this.patientId' = field, 'patientId' = param

// Always true on admission

// 2 params – quick intake  
// this() delegates to 4-arg constructor – must be first line

// Returns Patient, not void

// Returns the SAME object – enables chaining

// Same pattern

## ✓ Answer — Main.java

```
Patient p1 = new Patient(1001, "Alice Brown", 35, "ICU");
p1.printInfo();

Patient p2 = new Patient(1002, "Bob Perera");

p2.setAge(47).setWard("Cardiology").printInfo();
```

// Full constructor – all values set

// Short constructor – age=0, ward=General initially

// Method chain – each call returns 'this' (the p2 object)

## 📖 Line-by-Line Explanation

| Line / Concept                                               | Detailed Explanation                                                                                                                                                                                                                              |
|--------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>this.patientId = patientId</code>                      | Without 'this.', the statement would be <code>patientId = patientId</code> — assigning the parameter to itself. The field would never be updated. 'this.' explicitly targets the instance variable.                                               |
| <code>this.isAdmitted = true</code>                          | 'this' is optional here (no name clash) but makes the code readable — clearly setting the current object's field. The field is set to true for every new Patient, without the caller needing to specify it.                                       |
| <code>this(patientId, name, 0, "General")</code>             | Constructor chaining — one constructor calling another in the same class using <code>this()</code> . Must always be the very first statement. Avoids duplicating the assignment code. Defaults <code>age=0</code> and <code>ward=General</code> . |
| <code>public Patient setAge(int age)</code>                  | The return type is Patient, not void. This is what enables chaining. Each setter modifies the field and then returns the same object.                                                                                                             |
| <code>return this</code>                                     | Returns a reference to the current object. The caller gets back the exact same Patient, so the next method call in the chain operates on the same object.                                                                                         |
| <code>p2.setAge(47).setWard("Cardiology").printInfo()</code> | <code>setAge(47)</code> runs, updates age, returns p2. <code>.setWard("Cardiology")</code> runs on p2, updates ward, returns p2. <code>.printInfo()</code> runs on p2. All three calls are on the same object.                                    |

## Q5

## Hospital Constants & Staff Counter - static and final

### Background / Scenario

The hospital management system needs two types of shared information:

**Constants:** The hospital name and the maximum ward capacity are fixed values that never change and must be available without creating any object.

**Class-level counter:** The system must track how many Staff objects have ever been created. This count is not per-object - it belongs to the class itself.

*A static method `displayHospitalInfo()` must print the hospital name and the total staff count. It must work even before any Staff object is created.*

### What You Must Do

#### Tasks

- ▶ Create a Staff class with a public static final String `HOSPITAL_NAME = "City General Hospital"` and a public static final int `MAX_WARD_CAPACITY = 30`.
- ▶ Add a private static int `staffCount = 0` variable. (Increment it inside every constructor so it tracks the total number of Staff objects created.)
- ▶ Add instance fields: `staffId` (String), `staffName` (String), `department` (String).
- ▶ Write a constructor that takes all three instance fields and increments `staffCount`.
- ▶ Write a static method `displayHospitalInfo()` that prints `HOSPITAL_NAME` and `staffCount`. This method must NOT access instance fields.
- ▶ Write an instance method `displayStaff()` that prints the staff member's details.
- ▶ In Main: call `Staff.displayHospitalInfo()` before creating any object. Create three Staff objects. Call `Staff.displayHospitalInfo()` again and observe the count has changed.

#### Expected Output

```
--- Hospital Info ---
Hospital      : City General Hospital
Max Capacity  : 30 per ward
Staff On Record: 0

--- Hospital Info ---
Hospital      : City General Hospital
Max Capacity  : 30 per ward
Staff On Record: 3

[S001] Dr. Nimal Fernando - Cardiology
[S002] Nurse Priya Perera - ICU
[S003] Dr. Amara Silva    - Neurology
```

## ✓ Answer — Staff.java

```
public class Staff {

    // — Class-level constants —————
    public static final String HOSPITAL_NAME    = "City General
Hospital";
    public static final int    MAX_WARD_CAPACITY = 30;

    // — Class-level counter —————
    private static int staffCount = 0;

    // — Instance fields —————
    private String staffId;

    private String staffName;
    private String department;

    public Staff(String staffId, String staffName, String
department) {
        this.staffId    = staffId;
        this.staffName   = staffName;
        this.department  = department;
        staffCount++;
    }

    // — Static utility method —————
    public static void displayHospitalInfo() {

        System.out.println("--- Hospital Info ---");
        System.out.println("Hospital      : " + HOSPITAL_NAME);

        System.out.println("Max Capacity : " + MAX_WARD_CAPACITY
+ " per ward");
        System.out.println("Staff On Record: " + staffCount);
    }

    public void displayStaff() {

        System.out.println("[ " + staffId + " ] " + staffName + " -
" + department);
    }
}
```

// static: one copy for the class. final: never changes  
// Naming convention for constants: ALL\_CAPS

// One shared copy – not per object

// Each Staff object has its OWN copy of these

// Runs every time a new Staff object is created

// static: called on class, no object needed

// Can access static fields directly

// Instance method – needs a specific object



## ✓ Answer — Main.java

```
Staff.displayHospitalInfo(); // Called on CLASS – no
                             // object created yet.
                             // staffCount = 0

    Staff s1 = new Staff("S001", "Dr. Nimal Fernando",
"Cardiology");
    Staff s2 = new Staff("S002", "Nurse Priya Perera",
"ICU");
    Staff s3 = new Staff("S003", "Dr. Amara Silva",
"Neurology");

    Staff.displayHospitalInfo(); // Now staffCount = 3 –
                                // shared by all objects

    s1.displayStaff();
    s2.displayStaff();
    s3.displayStaff();
```

## 📖 Line-by-Line Explanation

| Line / Concept                                        | Detailed Explanation                                                                                                                                                                                                                                                                                   |
|-------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>public static final String HOSPITAL_NAME</code> | <code>public</code> — accessible everywhere. <code>static</code> — belongs to the class, not any instance. <code>final</code> — the value can never be reassigned after initialisation. Together these three modifiers create a true constant. By convention, constants use ALL_CAPS with underscores. |
| <code>private static int staffCount = 0</code>        | <code>static</code> means one copy in memory, shared across every <code>Staff</code> object. It lives in the method area of the JVM (not on the heap with objects). <code>private</code> means only <code>Staff</code> code can modify it directly.                                                    |
| <code>staffCount++</code>                             | This line is inside the constructor, which runs every time 'new Staff(...)' is called. Because <code>staffCount</code> is static, every new object increments the same counter — it is not reset per object.                                                                                           |
| <code>public static void displayHospitalInfo()</code> | static method — invoked as <code>Staff.displayHospitalInfo()</code> . No 'this' exists inside a static method. It cannot access instance fields ( <code>staffId</code> , <code>staffName</code> ) because there is no specific object.                                                                 |
| <code>Staff.displayHospitalInfo()</code> before new   | The method is called before any object exists. This is only possible because it is static — it belongs to the class itself, not to any instance.                                                                                                                                                       |
| <code>HOSPITAL_NAME</code> inside static method       | Static methods can access static fields directly (they are at the same level — class level). They cannot access instance-level fields.                                                                                                                                                                 |

## Q6

## Salary Calculation — Abstraction & Runtime Polymorphism

### Background / Scenario

City General Hospital employs two types of medical staff whose salaries are calculated differently:

**Doctor:** Monthly salary = baseSalary + (consultationFee × numberOfConsultations)

**Nurse:** Monthly salary = baseSalary + (overtimeHours × 15.0)

The HR system must process all staff in a single loop without needing to know whether each staff member is a Doctor or a Nurse. The correct salary formula must be called automatically for each type. This is the core use case for abstraction and runtime polymorphism.

### What You Must Do

#### Tasks

- ▶ Create an abstract class `MedicalStaff` with: protected fields `staffId`, `name`, `baseSalary`; an abstract method `calculateSalary()`; a concrete method `printPayslip()` that calls `calculateSalary()` and prints the result.
- ▶ Create class `Doctor` extending `MedicalStaff`. Add fields: `consultationFee` (double), `numberOfConsultations` (int). Implement `calculateSalary()` using the Doctor formula.
- ▶ Create class `Nurse` extending `MedicalStaff`. Add field: `overtimeHours` (int). Implement `calculateSalary()` using the Nurse formula.
- ▶ In Main, create a `MedicalStaff` array of size 4. Add 2 Doctors and 2 Nurses. Loop through the array and call `printPayslip()` on each. Java must automatically call the correct `calculateSalary()` for each type.
- ▶ Try: `MedicalStaff ms = new MedicalStaff(...)` — it must not compile. Explain why.

#### Expected Output

```
=== Payslip: Dr. Nimal Fernando ===
Staff ID   : D001
Base       : $5000.0
Total      : $8000.0

=== Payslip: Dr. Priya Silva ===
Staff ID   : D002
Base       : $5500.0
Total      : $8500.0

=== Payslip: Nurse Ana Mendis ===
Staff ID   : N001
Base       : $3000.0
Total      : $3600.0

=== Payslip: Nurse Kumari Perera ===
Staff ID   : N002
Base       : $2800.0
Total      : $3325.0
```

## ✓ Answer — MedicalStaff.java

```
public abstract class MedicalStaff {  
    protected String staffId;  
    protected String name;  
    protected double baseSalary;  
  
    public MedicalStaff(String staffId, String name, double  
baseSalary) {  
        this.staffId    = staffId;  
        this.name        = name;  
        this.baseSalary = baseSalary;  
    }  
  
    public abstract double calculateSalary();  
  
    public void printPayslip() {  
        System.out.println("=== Payslip: " + name + " ===");  
        System.out.println("Staff ID   : " + staffId);  
        System.out.println("Base      : $" + baseSalary);  
        System.out.println("Total     : $" +  
calculateSalary());  
    }  
}
```

// abstract: cannot be  
instantiated directly

// No body – EVERY non-abstract  
subclass MUST implement this

// Concrete method – inherited  
by Doctor and Nurse

// calculateSalary() here –  
which version? Decided at  
RUNTIME

## ✓ Answer — Doctor.java & Nurse.java

```
public class Doctor extends MedicalStaff {  
    private double consultationFee;  
    private int    numberOfConsultations;  
  
    public Doctor(String id, String name, double base,  
double fee, int consultations) {  
        super(id, name, base);  
  
        this.consultationFee    = fee;  
        this.numberOfConsultations = consultations;  
    }  
  
    @Override  
    public double calculateSalary() {  
        return baseSalary + (consultationFee *  
numberOfConsultations);  
    }  
}
```

// Calls MedicalStaff  
constructor

// Doctor formula

```

public class Nurse extends MedicalStaff {
    private int overtimeHours;

    public Nurse(String id, String name, double base, int
overtime) {
        super(id, name, base);
        this.overtimeHours = overtime;
    }

    @Override
    public double calculateSalary() {

        return baseSalary + (overtimeHours * 15.0);
    }
}

```

```

// Nurse formula – different
logic, same method name

```

## ✓ Answer — Main.java

```

MedicalStaff[] staff = new MedicalStaff[4];

    staff[0] = new Doctor("D001", "Dr. Nimal Fernando",
5000, 300, 10);
    staff[1] = new Doctor("D002", "Dr. Priya Silva",
5500, 250, 12);
    staff[2] = new Nurse ("N001", "Nurse Ana Mendis",
3000, 40);
    staff[3] = new Nurse ("N002", "Nurse Kumari Perera",
2800, 35);

    for (MedicalStaff s : staff) {

        s.printPayslip();

        System.out.println("");
    }

```

```

// Array declared as parent
type

```

```

// Loop sees each element as
MedicalStaff
// Java checks real type at
runtime – calls correct
calculateSalary()

```

## 📖 Line-by-Line Explanation

| Line / Concept                                        | Detailed Explanation                                                                                                                                                                                                         |
|-------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>public abstract class MedicalStaff</code>       | abstract class cannot be instantiated. Writing <code>new MedicalStaff()</code> causes a compile-time error. It exists only to be extended. It can have concrete methods, abstract methods, and fields — unlike an interface. |
| <code>public abstract double calculateSalary()</code> | No method body — ends with a semicolon. This is a contract: any class that extends <code>MedicalStaff</code> and is not itself abstract MUST provide a concrete implementation. Forgetting it causes a compile error.        |

|                                                             |                                                                                                                                                                                                                                                                                                                                                                                                               |
|-------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>calculateSalary() in<br/>printPayslip()</code>        | This is the key line. <code>printPayslip()</code> is defined in <code>MedicalStaff</code> and calls <code>calculateSalary()</code> . At compile time the compiler only knows it is <code>MedicalStaff</code> . At runtime, if the actual object is a <code>Doctor</code> , <code>Doctor</code> 's version runs. If it is a <code>Nurse</code> , <code>Nurse</code> 's version runs. This is dynamic dispatch. |
| <code>MedicalStaff[] staff =<br/>new MedicalStaff[4]</code> | Array whose element type is <code>MedicalStaff</code> . <code>Doctor</code> and <code>Nurse</code> objects can be stored because they are both <code>MedicalStaff</code> . This is called an array of polymorphic references.                                                                                                                                                                                 |
| <code>for (MedicalStaff s :<br/>staff)</code>               | Each element is accessed as <code>MedicalStaff</code> . The compiler only allows calls to methods defined in <code>MedicalStaff</code> . But at runtime, Java resolves the actual type and calls the correct override.                                                                                                                                                                                        |
| <code>s.printPayslip() in loop</code>                       | When <code>s</code> holds a <code>Doctor</code> , <code>printPayslip()</code> internally calls <code>Doctor.calculateSalary()</code> . When <code>s</code> holds a <code>Nurse</code> , it calls <code>Nurse.calculateSalary()</code> . No <code>if/else</code> needed in the loop — polymorphism handles it.                                                                                                 |

## Q7

## Protecting Patient Records — Encapsulation

### Background / Scenario

A security audit has flagged the Patient class: fields are accessible directly from outside, meaning any code anywhere in the system can set a patient's age to -5 or blank out their name. This is a data integrity failure.

The audit team requires the following rules to be enforced at all times:

**Name:** Cannot be null or empty. If an invalid name is provided, throw an `IllegalArgumentException`.

**Age:** Must be between 0 and 120 inclusive. If invalid, print an error and leave the existing age unchanged.

**Ward:** Cannot be null or empty. If invalid, default to 'General' and print a warning.

*The constructor must call the setters (not assign directly) so that validation runs during object creation too — not just on later updates.*

### What You Must Do

#### Tasks

- ▶ Make ALL fields of Patient private.
- ▶ Write getters for all fields (`getPatientId`, `getName`, `getAge`, `getWard`, `isAdmitted`).
- ▶ Write a `setName(String)` that throws `IllegalArgumentException` if name is null or blank.
- ▶ Write a `setAge(int)` that validates 0–120. On failure: print error, do NOT update the field.
- ▶ Write a `setWard(String)` that defaults to 'General' on invalid input with a printed warning.
- ▶ In the constructor, call `setName()`, `setAge()`, `setWard()` instead of assigning directly.
- ▶ In Main: create a valid patient. Then attempt `setAge(-10)`, `setAge(150)`, `setWard("")`. Print the fields after each attempt to show the original values are protected.

#### Expected Output

```
Created: Alice Brown, Age: 35, Ward: ICU
ERROR: Invalid age -10. Age unchanged. Still: 35
ERROR: Invalid age 150. Age unchanged. Still: 35
WARNING: Invalid ward. Defaulting to General.
After attempts: Alice Brown, Age: 35, Ward: General
```

### Answer — Patient.java (fully encapsulated)

```
public class Patient {
    private int    patientId;

    private String name;
    private int    age;
    private String ward;
    private boolean isAdmitted;
}
```

// ALL fields private – only accessible inside this class

```

    public Patient(int patientId, String name, int age, String
ward) {
        this.patientId = patientId;

        this.isAdmitted = true;
        setName(name);

        setAge(age);
        setWard(ward);
    }

```

```

// ID has no setter – set
once, immutable

```

```

// Calls setter –
validation runs here too

```

```

// — Getters —————
public int    getPatientId() { return patientId; }
public String getName()     { return name;      }
public int    getAge()      { return age;       }
public String getWard()     { return ward;      }
public boolean isAdmitted() { return isAdmitted;}

```

```

// — Setters with validation —————

```

```

public void setName(String name) {
    if (name == null || name.trim().isEmpty()) {

        throw new IllegalArgumentException("Name cannot be
null or blank.");
    }
    this.name = name;
}

```

```

// null check first, then
empty-string check

```

```

public void setAge(int age) {
    if (age < 0 || age > 120) {

        System.out.println("ERROR: Invalid age " + age + ".
Age unchanged. Still: " + this.age);
        return;
    }
    this.age = age;
}

```

```

// Range check

```

```

// Exit immediately – field
NOT updated

```

```

public void setWard(String ward) {
    if (ward == null || ward.trim().isEmpty()) {

        System.out.println("WARNING: Invalid ward. Defaulting
to General.");
        this.ward = "General";
        return;
    }
    this.ward = ward;
}
}

```

```

// Null or whitespace-only
string rejected

```

```

// Safe default applied

```

## ✓ Answer — Main.java

```
Patient p = new Patient(1001, "Alice Brown", 35, "ICU");
System.out.println("Created: " + p.getName() + ", Age: "
+ p.getAge() + ", Ward: " + p.getWard());

p.setAge(-10); // Invalid – prints error, age stays 35
p.setAge(150); // Invalid – prints error, age stays 35
p.setWard(""); // Invalid – defaults to General, prints warning

System.out.println("After attempts: " + p.getName()
+ ", Age: " + p.getAge() + ", Ward: " + p.getWard());
```

## 📖 Line-by-Line Explanation

| Line / Concept                                       | Detailed Explanation                                                                                                                                                                                                                          |
|------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>private int age</code>                         | <code>private</code> is the strictest access level. Code outside <code>Patient</code> cannot write <code>p.age = -10</code> directly. Every change must go through the validated setter. This is the fundamental guarantee of encapsulation.  |
| <code>setAge(age)</code> inside constructor          | The constructor calls the setter rather than assigning directly ( <code>this.age = age</code> ). This ensures the validation logic runs at object creation time, not just on later updates. A single source of truth for the validation rule. |
| <code>if (age &lt; 0    age &gt; 120)</code>         | Business rule enforced inside the class. The outside world hands data in; the class decides whether to accept it. This is sometimes called defensive programming.                                                                             |
| <code>return</code> (after error)                    | The early return exit pattern — also called a guard clause. If the condition fails, the method prints the error and exits before the assignment line is reached. The field is never updated with bad data.                                    |
| <code>this.ward = "General"</code>                   | When invalid data is provided, a safe default is applied rather than crashing the program. This is a design decision — production systems often log the warning and use a fallback, rather than throwing an exception.                        |
| <code>throw new IllegalArgumentException(...)</code> | For name, the consequence is more severe — an exception is thrown. The calling code must handle this or the program halts. Use this approach when the data is so critical that no default is appropriate.                                     |
| <code>public int getAge()</code>                     | A getter provides controlled read access. The caller can see the value but cannot change it without going through the validated setter. This separates reading from writing — single responsibility for each method.                          |



# Practice Questions

## Library Management System — Integrated Capstone

**Instructions for students:** This session builds a working Library Management System from scratch. You will apply every OOP concept studied in Sessions 1–11. Build the classes in the order given — each question depends on the previous. By the end you will have a fully running system.

**Project structure:** Create ONE IntelliJ project. Put each class in a separate .java file. Run Main.java at the end of each question to verify your progress.

### Q8 Catalogue Foundation — Abstract Class & Inheritance

#### Background / Scenario

The City Public Library catalogues two types of items: **Books** and **Magazines**. Both share **common attributes** (an item ID, a title, and the year published) but each has **its own unique details**.

**Book:** author (String), isbn (String), numberOfPages (int)

**Magazine:** issueNumber (int), publisher (String)

A polymorphic method **getItemInfo()** must return a formatted String describing the item. Because Books and Magazines describe themselves differently, this method is declared abstract in the parent and overridden in each child. The catalogue also needs to **track item availability** (checked out or on the shelf).

#### What You Must Do

##### Tasks

- ▶ Create abstract class `LibraryItem` with: protected fields `itemId`, `title`, `yearPublished`; private boolean `available` (initially true); abstract method `String getItemInfo()`; concrete methods `checkOut()`, `returnItem()`, `isAvailable()`.
- ▶ Create class `Book` extending `LibraryItem`. Add three private fields. Use `super()` in constructor. Override `getItemInfo()` to return a formatted string including `[BOOK]` tag and all fields.
- ▶ Create class `Magazine` extending `LibraryItem`. Add two private fields. Override `getItemInfo()` similarly with `[MAGAZINE]` tag.
- ▶ In `Main`, create two `Books` and one `Magazine`. Call `getItemInfo()` on all three. Check out one `Book` and call `getItemInfo()` again to see the availability change.

##### Expected Output

```
[BOOK]      | ID: B001 | Clean Code by Robert C. Martin | ISBN: 978-0132 | Pages: 464 |
Available: true
[BOOK]      | ID: B002 | Head First Java by Kathy Sierra | ISBN: 978-0596 | Pages: 688 |
Available: true
[MAGAZINE]   | ID: M001 | Tech Today Issue #12 | Publisher: TechPress | Available: true
```

After checkout of B001:  
[BOOK] | ID: B001 | Clean Code by Robert C. Martin | ISBN: 978-0132 | Pages: 464 |  
Available: false

#### ✓ Answer — LibraryItem.java

```
public abstract class LibraryItem {  
    protected String itemId;  
    protected String title;  
    protected int    yearPublished;  
    private boolean  available;  
  
    public LibraryItem(String itemId, String title, int  
year) {  
        this.itemId      = itemId;  
        this.title        = title;  
        this.yearPublished = year;  
        this.available    = true;  
    }  
  
    public abstract String getItemInfo();  
  
    public boolean isAvailable() { return available; }  
    public void checkOut()       { available = false; }  
    public void returnItem()     { available = true; }  
}
```

// Cannot instantiate – only extend  
// protected: subclasses can read directly  
// private: controlled only through checkOut/returnItem  
// Every item starts on the shelf  
// No body – each subclass defines its own format  
// Read-only access to private field  
// Marks item as borrowed  
// Marks item as back on shelf

#### ✓ Answer — Book.java

```
public class Book extends LibraryItem {  
    private String author;  
    private String isbn;  
    private int    numberOfPages;  
  
    public Book(String itemId, String title, int year,  
                String author, String isbn, int pages) {  
        super(itemId, title, year);  
        this.author      = author;  
        this.isbn        = isbn;  
        this.numberOfPages = pages;  
    }  
}
```

// IS-A LibraryItem  
// Pass common fields to parent

```
@Override
public String getItemInfo() {
    return "[BOOK]      | ID: " + itemId
        + " | " + title + " by " + author
        + " | ISBN: " + isbn
        + " | Pages: " + numberOfPages
        + " | Available: " + isAvailable();
}
}
```

// Implements the abstract contract

// itemId is protected – accessible here

// Call getter for private field

 Line-by-Line Explanation

| Line / Concept                       | Detailed Explanation                                                                                                                                                                                                                     |
|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| private boolean available            | Even Book and Magazine cannot set available directly because it is private to LibraryItem. They can call isAvailable(), checkOut(), returnItem() — which is correct: they don't change availability themselves, the Library system does. |
| public abstract String getItemInfo() | Forces every subclass to define its own format. If Book forgets to implement it, Book would also have to be declared abstract — meaning no Book objects could be created. The compiler enforces this.                                    |
| super(itemId, title, year) in Book   | Passes the three shared fields up to LibraryItem's constructor. Without this, itemId, title, yearPublished would be null/0 because Book's constructor cannot directly assign protected fields through a parent constructor.              |
| itemId inside Book.getItemInfo()     | itemId is protected in LibraryItem, so Book can access it directly. If it were private, Book would need to call a getter like getItemId(). This is why the choice between private and protected matters in inheritance hierarchies.      |
| isAvailable() in getItemInfo()       | available is private in LibraryItem — Book cannot read it as this.available. It must call isAvailable() which is a public method — this is encapsulation within the inheritance hierarchy.                                               |

## Q9

## Member Records — Encapsulation, static, final & this

### Background / Scenario

The library issues membership cards. A Member has a `memberId`, `name`, `email`, and a `list of borrowed item IDs`. Business rules enforced at all times:

**Rule 1:** A member may borrow at most `MAX_BORROW_LIMIT = 5` items at a time.

**Rule 2:** `memberId` is assigned at creation and can **NEVER** be changed afterwards.

**Rule 3:** Name cannot be null or empty.

**Rule 4:** A **class-level counter** tracks the total number of Member objects ever created.

A short constructor (`id`, `name` only) must exist for quick registration, defaulting email to 'Not provided'. It must chain to the full constructor using `this()`.

### What You Must Do

#### Tasks

- ▶ Declare: `public static final int MAX_BORROW_LIMIT = 5` and `private static int memberCount = 0`.
- ▶ Declare `private final String memberId` — `final` means it cannot be reassigned after the constructor.
- ▶ Write full constructor (`id`, `name`, `email`): use `setName()` for validation, set email directly, initialise `borrowedItems` as `new ArrayList<String>()`, increment `memberCount`.
- ▶ Write short constructor (`id`, `name`) that calls `this(id, name, "Not provided")`.
- ▶ Write `borrowItem(String itemId)`: returns `boolean`. Adds item only if current list size < `MAX_BORROW_LIMIT`.
- ▶ Write `returnItem(String itemId)`: removes the item from the list.
- ▶ Write `displayMember()` that prints all fields and the borrowed list.
- ▶ Write `static getMemberCount()` returning `memberCount`.

### Answer — Member.java

```
import java.util.ArrayList;

public class Member {
    public static final int MAX_BORROW_LIMIT = 5;

    private static int memberCount = 0;

    private final String memberId;

    private String name;
    private String email;
```

```
// Import ArrayList from
java.util package

// Constant – class-level, never
changes
// Shared counter – one copy for
all Member objects

// final instance field – set
once, never reassigned
```

```

private ArrayList<String> borrowedItems;

// — Full Constructor —————
public Member(String memberId, String name, String email)
{
    this.memberId = memberId;

    setName(name);

    this.email = (email != null) ? email : "Not
provided";
    borrowedItems = new ArrayList<>();

    memberCount++;
}

// — Short Constructor —————
public Member(String memberId, String name) {
    this(memberId, name, "Not provided");
}

// — Validated Setter —————
public void setName(String name) {
    if (name == null || name.trim().isEmpty())
        throw new IllegalArgumentException("Name cannot
be blank.");
    this.name = name;
}

// — Borrow / Return —————
public boolean borrowItem(String itemId) {
    if (borrowedItems.size() < MAX_BORROW_LIMIT) {
        borrowedItems.add(itemId);
        return true;
    }
    System.out.println(name + " has reached the borrow
limit of " + MAX_BORROW_LIMIT);
    return false;
}

public void returnItem(String itemId) {
    borrowedItems.remove(itemId);
}

// — Display —————
public void displayMember() {
    System.out.println("Member ID : " + memberId);
    System.out.println("Name      : " + name);
}

```

```

// List of item IDs currently
borrowed

```

```

// final field – can only be
assigned here
// Validation runs at creation
time

```

```

// Empty list – no items
borrowed yet
// Increments the shared class-
level counter

```

```

// Chains to full constructor –
avoids duplication

```

```

// Check limit before adding

```

```

// Signal success

```

```

// Signal failure – caller can
handle

```

```

// ArrayList.remove(String)
removes first matching element

```

```

        System.out.println("Email      : " + email);
        System.out.println("Borrowed   : " +
borrowedItems.size() + "/" + MAX_BORROW_LIMIT);
        System.out.println("Items      : " + borrowedItems);
    }

    // — Getters & Static method —————
    public String  getMemberId()      { return memberId;
}
    public String  getName()          { return name;
}
    public static int getMemberCount() { return
memberCount;}
}

```

```

// Static method – call as
Member.getMemberCount()

```

## Line-by-Line Explanation

| Line / Concept                                            | Detailed Explanation                                                                                                                                                                                                                               |
|-----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>public static final int MAX_BORROW_LIMIT</code>     | <code>public</code> — readable anywhere. <code>static</code> — one copy per class, not per object. <code>final</code> — cannot be reassigned. The ALL_CAPS name signals 'this is a constant'. Access as <code>Member.MAX_BORROW_LIMIT</code> .     |
| <code>private static int memberCount</code>               | <code>static</code> makes this a class-level variable. One copy exists in memory, shared by all <code>Member</code> objects. It starts at 0 before any object is created.                                                                          |
| <code>private final String memberId</code>                | <code>final</code> instance variable — can only be assigned once in the constructor. After the constructor exits, even code inside <code>Member</code> cannot write <code>this.memberId = something</code> . Guarantees IDs are permanently fixed. |
| <code>this.memberId = memberId</code><br>(in constructor) | The only place a <code>final</code> instance variable can be assigned. The right-hand side is the constructor parameter. After this line, <code>memberId</code> is frozen.                                                                         |
| <code>memberCount++</code> in constructor                 | The constructor runs every time 'new <code>Member(...)</code> ' is called. Because <code>memberCount</code> is <code>static</code> , all instances share it. Three new <code>Member</code> calls → <code>memberCount</code> becomes 3.             |
| <code>this(memberId, name, "Not provided")</code>         | Constructor chaining — calls the 3-arg constructor from the 2-arg constructor. Avoids duplicating the assignment code. The default email is supplied automatically.                                                                                |
| <code>borrowedItems.size() &lt; MAX_BORROW_LIMIT</code>   | Business rule enforced inside the class. The <code>Library</code> class that calls <code>borrowItem()</code> cannot override this limit because <code>borrowedItems</code> is <code>private</code> . Encapsulation protects the invariant.         |
| <code>public static int getMemberCount()</code>           | A static getter — called as <code>Member.getMemberCount()</code> . No <code>Member</code> object is needed. Reports class-level state. Cannot reference instance fields because it is <code>static</code> .                                        |

## Q10

## Complete Library System — Polymorphism & Full Integration

### Background / Scenario

All the building blocks are ready. Now build the Library class that brings everything together. The library maintains a catalogue of up to 100 LibraryItem objects (polymorphic — can be Books or Magazines) and a list of up to 50 Members.

The Library must support these operations:

**addItem(LibraryItem):** Add any LibraryItem subtype to the catalogue.

**addMember(Member):** Register a new member.

**checkoutItem(String itemId, String memberId):** Find the item and member by ID. If item is available and member is within the borrow limit, check it out.

**returnItem(String itemId, String memberId):** Return a borrowed item.

**listAllItems():** Print getItemInfo() for every item — Java automatically calls the correct overriding version for each type.

*In Main, add 3 Books and 2 Magazines. Register 2 Members. Check out 2 items for member 1. Try checking out the same item with member 2 — it should be unavailable. Return one item and check it out again.*

### Answer — Library.java

```
public class Library {
    public static final String LIBRARY_NAME = "City Public Library";
    private static final int MAX_ITEMS = 100;
    private static final int MAX_MEMBERS = 50;

    private LibraryItem[] items;
    private Member[] members;
    private int itemCount = 0;
    private int memberCount = 0;

    public Library() {
        items = new LibraryItem[MAX_ITEMS];
        members = new Member[MAX_MEMBERS];
    }

    // — Add Item —————
    public boolean addItem(LibraryItem item) {
        if (item == null || itemCount >= MAX_ITEMS) return false;
        items[itemCount++] = item;
        return true;
    }
}
```

// Polymorphic array – holds Book or Magazine

// Parameter type = parent. Accepts Book or Magazine

```

// — Add Member —————
public boolean addMember(Member member) {
    if (member == null || memberCount >= MAX_MEMBERS) return
false;
    members[memberCount++] = member;
    return true;
}

```

```

// — Find helpers —————
private LibraryItem findItem(String itemId) {

```

```

// private – internal
helper, not part of public
API

```

```

    for (int i = 0; i < itemCount; i++)
        if (items[i].itemId.equals(itemId)) return items[i];
    return null;
}

```

```

private Member findMember(String memberId) {
    for (int i = 0; i < memberCount; i++)
        if (members[i].getMemberId().equals(memberId)) return
members[i];
    return null;
}

```

```

// — Check Out —————
public void checkOutItem(String itemId, String memberId) {
    LibraryItem item    = findItem(itemId);
    Member      member = findMember(memberId);

```

```

    if (item == null) { System.out.println("Item not found:
" + itemId); return; }
    if (member == null) { System.out.println("Member not
found: " + memberId); return; }

```

```

    if (!item.isAvailable()) {

        System.out.println("'" + item.title + "' is already
checked out.");
        return;
    }

```

```

// Encapsulated check –
item manages its own state

```

```

    if (member.borrowItem(itemId)) {

        item.checkOut();

```

```

// Member enforces its own
borrow limit

```

```

// Item marks itself
unavailable

```

```

        System.out.println(member.getName() + " checked out '"
+ item.title + "'");
    }
}

```

```

// — Return Item —————
public void returnItem(String itemId, String memberId) {
    LibraryItem item    = findItem(itemId);

```



```

        Member member = findMember(memberId);
        if (item == null || member == null) {
System.out.println("Return failed."); return; }
        member.returnItem(itemId);

        item.returnItem();

        System.out.println(member.getName() + " returned '" +
item.title + "'");
    }

    // — List All Items —————
    public void listAllItems() {
        System.out.println("\n=== " + LIBRARY_NAME + " Catalogue
===");
        for (int i = 0; i < itemCount; i++)
            System.out.println(items[i].getItemInfo());

        System.out.println("");
    }
}

```

// Remove from member's borrowed list

// Mark item as available again

// Polymorphism: Book prints Book info, Magazine prints Magazine info

#### ✓ Answer — Main.java (Full Workflow)

```

public class Main {
    public static void main(String[] args) {
        Library lib = new Library();

        // Add items
        lib.addItem(new Book("B001","Clean Code",      2008,
"Robert C. Martin","978-0132",464));
        lib.addItem(new Book("B002","Head First Java",  2003,
"Kathy Sierra",    "978-0596",688));
        lib.addItem(new Book("B003","Effective Java",   2018,
"Joshua Bloch",    "978-0134",412));
        lib.addItem(new Magazine("M001","Tech Today",   2024,
12, "TechPress"));
        lib.addItem(new Magazine("M002","Dev World",    2024,
5, "CodeMedia"));

        // Register members
        Member m1 = new Member("MEM001", "Alice",
"alice@mail.com");
        Member m2 = new Member("MEM002", "Bob");

        lib.addMember(m1);
        lib.addMember(m2);

        lib.listAllItems();

        // Checkout & return workflow
    }
}

```

// Short constructor – email defaults

```
lib.checkOutItem("B001", "MEM001");           // Alice borrows Clean Code
lib.checkOutItem("M001", "MEM001");           // Alice borrows Tech Today
lib.checkOutItem("B001", "MEM002");           // Bob tries – already out

lib.listAllItems();                           // B001 and M001 now show
                                              Available: false

lib.returnItem("B001", "MEM001");             // Alice returns Clean Code
lib.checkOutItem("B001", "MEM002");           // Bob can now borrow it

System.out.println("\nTotal members registered: " +
Member.getMemberCount());
m1.displayMember();
}
}
```

 Line-by-Line Explanation

| Line / Concept                                           | Detailed Explanation                                                                                                                                                                                                                                                                                                                                        |
|----------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>private LibraryItem[] items</code>                 | The array type is <code>LibraryItem</code> (abstract parent). It can hold any <code>Book</code> or <code>Magazine</code> object. This is the core of polymorphism — one container for all item types.                                                                                                                                                       |
| <code>public boolean addItem(LibraryItem item)</code>    | The parameter type is <code>LibraryItem</code> . When you call <code>addItem(new Book(...))</code> , Java accepts it because <code>Book</code> IS-A <code>LibraryItem</code> . The same method works for <code>Magazine</code> . No overloading needed.                                                                                                     |
| <code>private LibraryItem findItem(String itemId)</code> | <code>private</code> — this is an internal helper. It is not part of the library's public interface. Only <code>checkOutItem()</code> and <code>returnItem()</code> need it. Hiding internal helpers is good encapsulation.                                                                                                                                 |
| <code>items[i].itemId.equals(itemId)</code>              | <code>itemId</code> is protected in <code>LibraryItem</code> , so <code>Library</code> (in the same package) can access it directly. <code>.equals()</code> is used instead of <code>==</code> because <code>String</code> comparison in Java must use <code>.equals()</code> for content equality.                                                         |
| <code>if (member.borrowItem(itemId))</code>              | The borrow limit is enforced inside <code>Member</code> — <code>Library</code> does not check the limit itself. Each class is responsible for enforcing its own rules. This is called the Single Responsibility Principle.                                                                                                                                  |
| <code>item.checkOut()</code>                             | The availability state is managed inside <code>LibraryItem</code> . <code>Library</code> just calls the method — it does not set <code>available = false</code> directly (that field is private). Encapsulation is maintained across classes.                                                                                                               |
| <code>items[i].getItemInfo() in listAllItems()</code>    | <code>items[i]</code> is declared as <code>LibraryItem</code> . At runtime, if slot <code>i</code> holds a <code>Book</code> , <code>Book.getItemInfo()</code> runs and prints book-specific info. If it holds a <code>Magazine</code> , <code>Magazine.getItemInfo()</code> runs. Java resolves this at runtime — dynamic dispatch / runtime polymorphism. |
| <code>Member.getMemberCount()</code>                     | Static method called on the class name. No <code>Member</code> object needed. Reports the class-level <code>memberCount</code> which was incremented in the constructor of every <code>Member</code> created.                                                                                                                                               |